

Manual Técnico Combinado

Arquitectura **MVC** · **JDBC** · **Swing** — Caso práctico: Gestión de Libros

Índice

1. Introducción a la Arquitectura MVC
2. Gestión de Conexión a Base de Datos
3. Manejo de Excepciones (Personalizadas + Checked/Unchecked)
4. Modelo: Entidad **Libro** y **LibroDao** (CRUD)
5. Componentes JDBC y Manipulación de Datos
6. Vista (**frmLibros2**) y Lógica de Interfaz con Swing
7. Controlador (**LibroController**) y Manejo de Eventos
8. Funcionalidades Avanzadas de la JTable
9. Conclusión y Buenas Prácticas

1. Introducción a la Arquitectura MVC

El sistema implementa el patrón **Modelo–Vista–Controlador (MVC)** para garantizar la escalabilidad y mantenibilidad del software. Esta arquitectura permite un **desacoplamiento efectivo** entre la lógica de negocio y la interfaz de usuario, facilitando que la persistencia se gestione de forma independiente a su representación.

El objetivo arquitectónico es lograr **máxima cohesión y mínimo acoplamiento**.

Clases clave del sistema

Componente	Rol
Libro (Entidad / POJO)	Representación del dominio de datos. Encapsula atributos (id, ISBN, título, autor, editorial, estado de préstamo, fechas). Es el contenedor que fluye entre capas.
LibroDao (DAO)	<i>Data Access Object</i> . Encapsula <u>todas las operaciones CRUD</u> y las sentencias SQL mediante JDBC, aislando el acceso a datos.
LibroController	Orquestador y mediador. Procesa solicitudes de la vista, registra escuchadores de eventos y coordina las reglas de negocio con el DAO.
frmLibros2 (Vista)	Interfaz Swing. Solo renderiza información y captura eventos : no debe contener lógica de procesamiento de datos.
Main	Punto de entrada (<i>bootstrap</i>). Instancia componentes críticos, gestiona la conexión global e inicia el ciclo de vida de la aplicación.

2. Gestión de Conexión a Base de Datos

La comunicación con el motor de base de datos reside en la clase `DbConnection` (también vista como `Conexion`). El diseño centraliza la configuración técnica y el ciclo de vida de los objetos `Connection`.

Patrón Singleton: se utiliza para centralizar el acceso y optimizar el pool de conexiones, asegurando que no se creen instancias innecesarias.

Atributo / Método	Propósito técnico
<code>db</code>	Instancia global de <code>DbConnection</code> que centraliza parámetros de red y credenciales.
<code>cn</code>	Objeto <code>Connection</code> que mantiene la sesión activa con el servidor.
<code>getConnection()</code>	Método que retorna la instancia de conexión, para ser inyectada en controladores y DAOs.

Nota técnica — URL JDBC: la URL debe seguir estrictamente el formato `jdbc:mysql://localhost:3306/nombre_bd`. Verifica los dos puntos tras el protocolo y el puerto; un error de sintaxis aquí impide la conexión.

Ejemplo de implementación con Singleton

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DbConnection {
    private static Connection cn = null;
    private final String base = "biblioteca";
    private final String url = "jdbc:mysql://localhost:3306/" + base;
    private final String user = "root";
    private final String pass = "password";

    public Connection getConnection() {
        try {
            // Verificamos si la conexión es nula o cerrada por timeout
            if (cn == null || cn.isClosed()) {
                Class.forName("com.mysql.cj.jdbc.Driver");
                cn = DriverManager.getConnection(url, user, pass);
            }
        } catch (ClassNotFoundException | SQLException e) {
            System.err.println("Error Crítico de Conexión: " + e.getMessage());
        }
        return cn;
    }
}

```

Eficiencia: las conexiones deben abrirse solo cuando sea estrictamente necesario. La verificación `cn == null || cn.isClosed()` evita abrir conexiones redundantes y reaprovecha la sesión vigente.

3. Manejo de Excepciones

3.1 Clasificación: Checked vs. Unchecked

Característica	Checked Exceptions	Unchecked Exceptions
Herencia	<code>Exception</code>	<code>RuntimeException</code>
Verificación	Obligatoria por el compilador	En tiempo de ejecución
Origen	Factores externos (red, E/S, SQL)	Errores de lógica del programador
Ejemplos	<code>SQLException</code> , <code>IOException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code>

3.2 Excepciones personalizadas del sistema

La robustez se fundamenta en **validación preventiva** antes de cualquier operación de escritura. Las excepciones personalizadas segmentan errores de infraestructura de errores de lógica de negocio:

- **`CamposVaciosException`**: se dispara durante la captura de datos si campos obligatorios carecen de información. Evita inconsistencias en la capa de datos.
- **`IsbnException`**: especializada en la integridad referencial y de formato del identificador único. Asegura que el ISBN cumpla los estándares definidos.

3.3 `throw` vs. `throws`

- **`throws`**: declaración en la firma del método; indica que puede propagar una excepción al llamante.
- **`throw`**: palabra reservada para disparar (lanzar) una instancia concreta de una excepción.

```
public void validarIsbn(String isbn) throws IsbnException { // declaración
    if (isbn == null || isbn.isBlank()) {
        throw new IsbnException("El ISBN no puede estar vacío"); // lanzamiento
    }
}
```

3.4 Control de flujo y recursos

`try-with-resources (Java 7+)`: automatiza el cierre de objetos `AutoCloseable` (`Connection`, `PreparedStatement`, `ResultSet`). Previene fugas de memoria (memory leaks) y reemplaza al clásico `try-catch-finally`.

3.5 Propagación hacia la interfaz

```
try {
    cargarDatos(sql); // puede lanzar SQLException o excepciones de negocio
} catch (SQLException e) {
    // Error crítico de infraestructura o sintaxis SQL
    e.printStackTrace();
    JOptionPane.showMessageDialog(null, "Error en la base de datos: " + e.getMessage());
} catch (IsbnException | CamposVaciosException e) {
    // Errores de validación controlados
    JOptionPane.showMessageDialog(null, e.getMessage());
}
```

Prohibido silenciar excepciones: un bloque `catch` vacío es deuda técnica peligrosa. Usa siempre `e.printStackTrace()` o logs descriptivos. Capturar `SQLException` es **imperativo** para diagnosticar fallos sin comprometer la estabilidad del hilo principal.

4. Modelo: Entidad `Libro` y `LibroDao` (CRUD)

4.1 Entidad `Libro` (POJO)

Refleja fielmente la estructura de la tabla `libro`:

```
public class Libro {
    private int idLibro;
    private String isbn;
    private String titulo;
    private String autor;
    private String editorial;
    private boolean prestado;
    private java.sql.Date fechaPrestamo;
    private java.sql.Date fechaDevolucion;

    public Libro() {}

    // Getters y Setters
    public int getIdLibro() { return idLibro; }
    public void setIdLibro(int idLibro) { this.idLibro = idLibro; }
    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getAutor() { return autor; }
    public void setAutor(String autor) { this.autor = autor; }
    public String getEditorial() { return editorial; }
    public void setEditorial(String e) { this.editorial = e; }
    public boolean isPrestado() { return prestado; }
    public void setPrestado(boolean p) { this.prestado = p; }
    public java.sql.Date getFechaPrestamo() { return fechaPrestamo; }
    public void setFechaPrestamo(java.sql.Date f) { this.fechaPrestamo = f; }
    public java.sql.Date getFechaDevolucion() { return fechaDevolucion; }
    public void setFechaDevolucion(java.sql.Date f) { this.fechaDevolucion = f; }
}
```

4.2 `LibroDao` — CRUD con `try-with-resources`

Buena práctica: usar `try-with-resources` garantiza el cierre automático de `Connection`, `PreparedStatement` y `ResultSet`. Es la forma estándar moderna de evitar fugas.

```

import java.sql.*;

public class LibroDao extends DbConnection {

    public boolean registrar(Libro l) {
        String sql = "INSERT INTO libro (isbn, titulo, autor, editorial, prestado) VALUES
    try (Connection cn = getConnection();
        PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setString(1, l.getIsbn());
            ps.setString(2, l.getTitulo());
            ps.setString(3, l.getAutor());
            ps.setString(4, l.getEditorial());
            ps.setBoolean(5, l.isPrestado());
            ps.execute();
            return true;
        } catch (SQLException e) {
            System.err.println("Error en Registro: " + e);
            return false;
        }
    }

    public boolean modificar(Libro l) {
        String sql = "UPDATE libro SET isbn=?, titulo=?, autor=?, editorial=?, prestado=?
    try (Connection cn = getConnection();
        PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setString(1, l.getIsbn());
            ps.setString(2, l.getTitulo());
            ps.setString(3, l.getAutor());
            ps.setString(4, l.getEditorial());
            ps.setBoolean(5, l.isPrestado());
            ps.setInt(6, l.getIdLibro());
            ps.execute();
            return true;
        } catch (SQLException e) {
            System.err.println("Error en Modificación: " + e);
            return false;
        }
    }

    public boolean eliminar(Libro l) {
        String sql = "DELETE FROM libro WHERE idLibro=?";
    try (Connection cn = getConnection();
        PreparedStatement ps = cn.prepareStatement(sql)) {
            ps.setInt(1, l.getIdLibro());
            ps.execute();
            return true;
        } catch (SQLException e) {
            System.err.println("Error en Eliminación: " + e);
            return false;
        }
    }
}

```



```
public boolean buscar(Libro l) {
    String sql = "SELECT * FROM libro WHERE isbn=?";
    try (Connection cn = getConnection();
        PreparedStatement ps = cn.prepareStatement(sql)) {
        ps.setString(1, l.getIsbn());
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                l.setIdLibro(rs.getInt("idLibro"));
                l.setIsbn(rs.getString("isbn"));
                l.setTitulo(rs.getString("titulo"));
                l.setAutor(rs.getString("autor"));
                l.setEditorial(rs.getString("editorial"));
                l.setPrestado(rs.getBoolean("prestado"));
                return true;
            }
            return false;
        }
    } catch (SQLException e) {
        System.err.println("Error en Búsqueda: " + e);
        return false;
    }
}
```

5. Componentes JDBC y Manipulación de Datos

La manipulación de datos se realiza con la API JDBC, transformando filas de una tabla relacional en objetos Java.

Componente	Función
<code>PreparedStatement</code>	Componente preferido para ejecutar SQL. Permite la <u>pre-compilación</u> de la consulta y protege contra <u>Inyección SQL</u> mediante parametrización (<code>?</code>).
<code>ResultSet</code>	Cursor / iterador sobre los resultados. Provee acceso a los datos crudos que luego se mapean a la entidad <code>Libro</code> .

5.1 Comparativa de métodos

Método	Uso	Devuelve
<code>executeQuery()</code>	Sentencias <code>SELECT</code>	Un <code>ResultSet</code> con los registros hallados
<code>executeUpdate()</code>	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	Un <code>int</code> con la cuenta de filas afectadas

5.2 Pasos de ejecución y *Data Mapping*

1. Apertura de conexión mediante `getConnection()` .
2. Instanciación del `PreparedStatement` con la cadena SQL.
3. Ejecución de la sentencia y obtención del `ResultSet` .
4. **Data Mapping**: recorrer el `ResultSet` y asignar cada columna a un atributo de una nueva instancia de `Libro` .
5. Adición del objeto `Libro` a una `List<Libro>` .
6. Cierre de recursos (asegurando el retorno de la lista al controlador).

6. Vista `frmLibros2` y Lógica de Interfaz con Swing

La clase `frmLibros2` representa la **capa de presentación**, construida sobre Swing. Su complejidad radica en la **sincronización entre el estado interno de los datos y los componentes visuales**.

6.1 Componentes clave

- `JTextField` — entrada de texto (ISBN, título, autor, editorial...).
- `JButton` — disparadores de acción (Guardar, Modificar, Eliminar, Navegación).
- `JTable` — visualización de colecciones de libros.
- `JCheckBox` — estado `prestado`.
- `JComboBox` + `JTextField` — filtrado dinámico de la consulta SQL.

Modificadores de acceso: los componentes del `JFrame` se declaran `public` para que el Controlador pueda asignar `ActionListener` y leer estados con `getText()` directamente.

6.2 Gestión de datos en memoria y ciclo de vida

El método `cargarDatos(String sql)` instancia el `LibroController` y la `DbConnection` para obtener la `List<Libro>`; inmediatamente después asigna `lc=null; db=null;` para que el **Garbage Collector** libere recursos de red y lógica una vez los libros están en memoria local.

6.3 Sincronización de la interfaz

- **DefaultTableModel (dtm):** modelo de tabla que actúa como punto de conexión entre la lista de objetos y el `JTable`. En `cargarGrid` transforma la lista en estructura matricial.
- **Navegación mediante puntero:** índice que gestiona el estado de navegación. Permite desplazarse por la lista **sin nuevas peticiones a la BD** (caching en cliente).

6.4 Componentes de eventos y navegación

Los escuchadores de `JButton` gestionan navegación y edición protegiendo siempre las **condiciones de borde**:

- **Primero / Atrás / Adelante / Último:** el método `mostrarLibros(int puntero)` actualiza `textIdLibros`, `textIsbn`, `textTitulo`, `textAutor`, `textEditorial`, `textFechaPrestamo` y el `JCheckBox chckbxPrestado`.
- **Gestión:** Nuevo, Guardar, Borrar — controlan el estado de edición y la persistencia final vía controlador.
- **Filtrado:** `comboCampo` + `textFiltrar` + `btnFiltrar` construyen dinámicamente la cláusula SQL que se envía a `cargarDatos`.

Seguridad de índices: los botones de navegación deben validar que `puntero ≥ 0` y `puntero ≤ libros.size() - 1` para evitar `IndexOutOfBoundsException`.

Nota técnica — Nulidad: al mostrar fechas, validar siempre antes de pintar: `if (libro.getFechaDevolucion() != null) { ... }`. Lo mismo aplica a `booleanos` del `CheckBox`.

7. Controlador `LibroController` y Manejo de Eventos

El `LibroController` es el **cerebro** que reacciona a las interacciones del usuario, registra los escuchadores y coordina vista ↔ modelo.

```
import java.awt.event.*;
import javax.swing.JOptionPane;

public class LibroController implements ActionListener {
    private Libro modelo;
    private LibroDao dao;
    private frmLibros2 vista;

    public LibroController(Libro modelo, LibroDao dao, frmLibros2 vista) {
        this.modelo = modelo;
        this.dao = dao;
        this.vista = vista;

        // Registro de escuchadores
        this.vista.btnGuardar.addActionListener(this);
        this.vista.btnModificar.addActionListener(this);
        this.vista.btnEliminar.addActionListener(this);
        this.vista.btnBuscar.addActionListener(this);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == vista.btnGuardar) {
            modelo.setIsbn(vista.textIsbn.getText());
            modelo.setTitulo(vista.textTitulo.getText());
            modelo.setAutor(vista.textAutor.getText());
            modelo.setEditorial(vista.textEditorial.getText());
            modelo.setPrestado(vista.chkcbxPrestado.isSelected());

            if (dao.registrar(modelo)) {
                JOptionPane.showMessageDialog(null, "Operación Exitosa");
            }
        }
        // Lógica similar para modificar, eliminar y buscar...
    }
}
```

8. Funcionalidades Avanzadas de la JTable

8.1 Búsqueda segura en tiempo real (`KeyReleased`)

Para búsqueda dinámica mientras el usuario escribe se usa el evento `KeyReleased`. Es **imperativo** usar `PreparedStatement` con `LIKE` para evitar Inyección SQL.

```
// En La capa DAO – búsqueda segura
public void buscarTiempoReal(String texto) {
    String sql = "SELECT * FROM libro WHERE titulo LIKE ?";
    try (Connection cn = getConnection();
        PreparedStatement ps = cn.prepareStatement(sql)) {
        ps.setString(1, "%" + texto + "%"); // parámetro seguro
        ResultSet rs = ps.executeQuery();
        // Cargar el DefaultTableModel con los resultados...
    } catch (SQLException e) { /* Manejo de error */ }
}
```

Nunca concatenar strings en SQL: usar siempre el placeholder `?` y `setString()`. La concatenación directa abre la puerta a inyección SQL.

8.2 Selección de fila y mapeo a campos

Cuando el usuario selecciona una fila para editar, el flujo técnico es:

1. Obtener el índice con `getSelectedRow()`.
2. Validar con `if (fila < 0)`.
3. Extraer valores por índice de columna (0 = ID, 1 = ISBN, 2 = Título...).

```
int fila = vista.tablaLibros.getSelectedRow();
if (fila < 0) {
    JOptionPane.showMessageDialog(null, "Debe seleccionar un registro de la tabla.");
} else {
    vista.textIdLibros.setText(vista.tablaLibros.getValueAt(fila, 0).toString());
    vista.textIsbn.setText(vista.tablaLibros.getValueAt(fila, 1).toString());
    vista.textTitulo.setText(vista.tablaLibros.getValueAt(fila, 2).toString());
    vista.textAutor.setText(vista.tablaLibros.getValueAt(fila, 3).toString());
    vista.textEditorial.setText(vista.tablaLibros.getValueAt(fila, 4).toString());
}
```

9. Conclusión y Buenas Prácticas

La arquitectura demuestra una **separación clara de responsabilidades**: el DAO maneja datos, el Controlador orquesta, y la Vista se limita a UX y estado local de navegación.

Buenas prácticas a memorizar

Principio	Aplicación
SEGURIDAD Sentencias preparadas	Usa <code>PreparedStatement</code> sistemáticamente. <u>Nunca concatenes strings en SQL.</u>
RECURSOS try-with-resources	No es opcional: previene fugas de conexiones que pueden colapsar el servidor de BD.
VALIDACIÓN En capas	Excepciones personalizadas (<code>IsbnException</code> , <code>CamposVaciosException</code>) detienen datos erróneos antes del motor de BD.
UI Validar selección	Antes de modificar o borrar, verifica que hay fila seleccionada en la <code>JTable</code> (<code>fila < 0</code>).
NULIDAD Campos sensibles	Validar fechas y booleanos antes de renderizar en la interfaz.
MEMORIA Cierre programático	Liberar conexiones y controladores tras la carga de datos (<code>lc = null; db = null; </code>).
PROHIBIDO Silenciar excepciones	Un <code>catch</code> vacío es deuda técnica. Siempre log o <code>printStackTrace()</code> .

Resumen mental para el examen:

- **MVC** = 3 capas desacopladas (Modelo / Vista / Controlador) + DAO.
- **JDBC** = `Connection` → `PreparedStatement` → `ResultSet` → mapeo a POJO.
- **Swing** = la vista solo renderiza; el controlador escucha eventos.
- **Singleton + try-with-resources** = pilares de la gestión de recursos.
- **Excepciones personalizadas** = validación temprana antes de tocar la BD.